

Hexapod Robot Project

STM32 + Zephyr RTOS + Raspberry Pi

Team Project Document

Version 1.0

Table of Contents

1.	Project Overview
2.	System Workflow
3.	System Architecture
4.	Servo Control System
5.	Inverse Kinematics & Gait
6.	Power System
7.	PS2 Controller
8.	Software Layout
9.	Testing & Debugging
10.	Team Structure & Assignments
11.	Execution Plan
12.	
13.	
14.	
15.	
16.	
17.	
18.	
19.	
20.	
21.	
22.	
23.	
24.	
25.	
26.	
27.	
28.	
29.	
30.	
31.	
32.	
33.	
34.	
35.	
36.	
37.	
38.	
39.	
40.	
41.	
42.	
43.	
44.	
45.	
46.	
47.	
48.	
49.	
50.	
51.	
52.	
53.	
54.	
55.	
56.	
57.	
58.	
59.	
60.	
61.	
62.	
63.	
64.	
65.	
66.	
67.	
68.	
69.	
70.	
71.	
72.	
73.	
74.	
75.	
76.	
77.	
78.	
79.	
80.	
81.	
82.	
83.	
84.	
85.	
86.	
87.	
88.	
89.	
90.	
91.	
92.	
93.	
94.	
95.	
96.	
97.	
98.	
99.	
100.	

1

Project Overview

What is a Hexapod Robot?

A hexapod is a robot with six legs. Because it always has at least three legs on the ground while walking, it is very stable — it does not tip over easily. This is the same reason ants and beetles are so good at walking on uneven surfaces.

Why 6 Legs?

6 legs allows a tripod gait: 3 legs lift and move forward while the other 3 push the body. This is the fastest stable walking pattern. It also means the robot can lose one leg and still walk (5 legs remain).

The Three Joints on Each Leg

Each leg has 3 joints. Each joint is driven by one servo motor.

Joint	Where on the Leg		What It Does	Human Equivalent
Coxa	Top	(hip) Upper	Rotates the leg left / right	Hip
Femur	segment	Lower	Lifts or lowers the leg	Thigh / knee
Tibia	segment		Extends or retracts the foot	Shin / ankle

Total servos: 6 legs x 3 joints = 18 servo motors.

What Each Component Does

Component	Simple Role	Why We Need It
STM32 Nucleo	Controls all robot movement	Runs real-time code for servos, gait, and IK at 50 Hz
Zephyr RTOS	Operating system on STM32	Lets STM32 run multiple jobs (threads) without missing deadlines
Raspberry Pi 4	Brain / AI co-processor	Handles camera, AI obstacle detection, autonomous navigation
PCA9685	Controls 18 servos via one I2C	Sends one message; chip generates 18 separate PWM signals
PS2 Controller	Manual user input	Joystick and buttons for human-controlled walking
3S LiPo Battery	Main power source (11.1V)	Lightweight rechargeable battery; 2 buck converters step down voltage

Key Rule: The STM32 + Zephyr is the LEAD controller. Every servo movement goes through the STM32. The Raspberry Pi assists but cannot directly control servos.

2

System Workflow

Every time the robot moves, these 6 steps happen in sequence, 50 times every second:

1	User gives input PushPS2joystickOR Raspberry Pi sends a movement command over UART.
2	STM32 reads the input Readsjoystickvaluesorparses the UART command. Decides direction and speed.
3	Gait Engine plans foot positions Softwaredecideswhichlegslift(swing)and which push (stance) right now.
4	IK calculates joint angles Foreachfoot's3Dtargetposition, math gives the 3 joint angles needed.
5	STM32 sends angles to PCA9685 over I2C All18anglesarentinonefastI2Cmessage.PCA9685 converts them to pulses.
6	Servo motors move — robot steps 18servosrotatetotheircommandedangles.Legs move. Robot walks.

Summary

Step	Action	Owner	Rate	100
1 2 3	Read user input	PS2 / RPi + STM32	x/sec	50
4 5 6	Plan foot targets	Gait Engine (STM32)	x/sec	50
	Calculate joint angles	IK Solver (STM32)	x/sec	50
	Send angles to PCA9685	STM32 over I2C	x/sec	
	Generate PWM signals	PCA9685 chip	Continuous	
	Servos rotate	18 servo motors	Continuous	

3

System Architecture

STM32 + Zephyr — Lead Controller

The STM32 is the primary controller. It runs Zephyr RTOS, which schedules five concurrent threads so that servo updates always happen on time. If the Raspberry Pi disconnects, the STM32 keeps the robot standing safely.

Raspberry Pi — Co-Processor

The RPi runs Linux (ROS2). It handles camera input, AI obstacle detection, and path planning. It communicates with the STM32 via UART (two wires, 1 Mbaud). It can only suggest movement — the STM32 decides whether to execute it.

STM32 + Zephyr RTOS (Lead)	Raspberry Pi 4 (Co-Processor)
Controls all 18 servo motors	Runs camera and AI detection
Executes gait patterns	Plans autonomous route
Solves inverse kinematics	Sends movement commands to STM32
Reads PS2 controller (SPI)	Provides SSH access and dashboard
Runs safety watchdog and E-stop	Logs telemetry data
Communicates with RPi over UART	Communicates with STM32 over UART

Zephyr RTOS Thread Structure

Priority	Thread	Job	Runs Every
(highest) 2	Servo Output	Send 18 angles to PCA9685 over I2C	20 ms 10
3 4 10	Input Handler	Read PS2 joystick and UART commands	ms 20 ms
(lowest)	Gait Engine	Calculate next foot positions	20 ms 100
	IK Solver	Convert foot positions to joint angles	ms
	Supervisor	Battery monitor, watchdog, USB logging	

Why This Matters: The servo output thread runs at priority 1 (highest). This guarantees servo updates are never delayed — even if other threads are busy.

4 Servo Control System

How a Servo is Controlled

Servo motors respond to PWM (Pulse Width Modulation) signals. The width of each pulse tells the servo what angle to move to. The signal repeats 50 times per second.

Servo Angle	Pulse Width	PCA9685 Count
0 degrees	1.0 ms 1.5	205
90 degrees	ms 2.0 ms	307 (neutral)
180 degrees		409

Why the PCA9685 Chip?

The STM32 does not have 18 PWM output pins available. The PCA9685 chip connects over just two wires (I2C: SDA + SCL) and generates 16 independent PWM signals on its own oscillator. The STM32 sends all 18 angles in one 73-byte I2C burst, taking about 1.6 ms.

Servo Channel Assignments

Channels	Leg	Joints Coxa,	Note Normal direction
0 - 2 3 - 5	Leg 1 Front-Left	Femur, Tibia Coxa,	Normal direction
6 - 8 9 -	Leg 2 Mid-Left	Femur, Tibia Coxa,	Normal direction Coxa
11 12 - 14	Leg 3 Rear-Left	Femur, Tibia Coxa,	inverted (mirror) Coxa
15 - 17	Leg 4 Front-Right	Femur, Tibia Coxa,	inverted (mirror) Coxa
	Leg 5 Mid-Right	Femur, Tibia Coxa,	inverted (mirror)
	Leg 6 Rear-Right	Femur, Tibia	

Calibration Note: Right-side legs (4-6) have their coxa direction inverted in firmware because they are mounted as a mirror image of left-side legs.

5 Inverse Kinematics & Gait

What is Inverse Kinematics (IK)?

IK answers: given a target foot position in 3D space (x, y, z), what angle should each of the 3 joints be? We use closed-form math (atan2 and the law of cosines) — no guessing, no iteration. All 6 legs solved in under 10 microseconds.

Step	Calculates	Formula
1 2 3	Horizontal reach to foot Straight-line	$\text{atan2}(y, x)$
4 5	distance: femur to foot Tibia angle	$L = \sqrt{x^2 + y^2} - \text{coxa_len}$
	Femur angle	$D = \sqrt{L^2 + \text{height}^2}$
		Law of cosines: D, femur, tibia
		atan2 + law of cosines offset

Gait Patterns

Gait	Legs Airborne	Speed	Stability	Use Case
Tripod	3 at once 2 at	Fast	Good	Default — flat ground
Ripple	once 1 at a	Medium	Better	Uneven surfaces
Wave	time	Slow	Maximum	Rough terrain / climbing

Tripod Gait Explained

Legs are split into Group A (legs 1, 4, 5) and Group B (legs 2, 3, 6). Group A swings forward while Group B pushes the body backward (propelling it forward). Then they swap. Each group uses a smooth half-arc trajectory so feet lift cleanly and land gently.

6 Power System

The robot runs on a 3S LiPo battery (11.1V). Two separate buck converters step this down to the two voltages needed. Servo motors and logic must never share the same power rail — motor spikes can crash the electronics.

Rail	Voltage	Min Current	Supplies
Logic rail	5 V 6 V	3 A	Raspberry Pi 4, STM32 Nucleo, PCA9685, PS2 receiver
Servo rail		10 A	All 18 servo motors

Critical: Add 4 x 1000 uF capacitors across the 6V servo rail. Without them, current spikes from servo motors will cause the STM32 to reset mid-walk.

Grounding Rule

All ground wires must meet at one single point (called the star point). Run separate wires from that point to: servo rail GND, logic rail GND, STM32 GND, and RPi GND. Never chain grounds through connectors. Bad grounding causes resets and UART errors.

Protection	Part	Purpose
Main fuse	10A automotive blade	Prevents fire on short circuit
Back-EMF	TVS diode 5.1V	Absorbs voltage spikes from motors
Capacitor bank	4 x 1000 uF / 10V	Absorbs servo inrush current
Low battery	STM32 ADC + code	Safe shutdown at 9.9V (3.3V/cell)

7 PS2 Controller

The PS2 controller uses a synchronous serial protocol at 250 kHz (similar to SPI). The STM32 polls it 100 times per second. Both devices run at 3.3V — no level shifting needed.

PS2 Pin	Signal	Connect To STM32 SPI2	Notes
Pin 1	DAT	MISO STM32 SPI2 MOSI	Data: controller to STM32
Pin 2	CMD	Star-point ground STM32	Commands: STM32 to controller
Pin 4	GND	3.3V STM32 GPIO (active	Must be common ground
Pin 5	VCC	low) STM32 SPI2 SCK	Power supply
Pin 6	CS		Pull low before each read
Pin 7	CLK		250 kHz clock from STM32

Important: The PS2 controller ships with analog mode OFF. Send the enable-analog command sequence (0x44) once on startup. Without this, joystick axes do not work.

Button Mapping

Input	Left	Robot Action	Notes
stick	X/Y	Walk direction	Forward / back / strafe
Right stick	X	Rotate body	Turn left or right in place
Triangle		Stand up	All legs to neutral / home position
Cross	Circle	Sit down	Safe resting position
Square	R1	Tripod gait	Switch to fast walk mode
L1	Start	Wave gait	Switch to slow stable mode
Select		Speed up	Increase step size
		Slow down	Decrease step size
		Emergency stop	Freeze all servos immediately
		Autonomous mode	Hand control to Raspberry Pi

8 Software Layout

STM32 / Zephyr Firmware

```
hexapod-firmware/ |-- prj.conf (enables I2C, SPI, UART, FPU) |-- src/ |-- main.c (creates
threads, starts timer) |-- drivers/ | |-- pca9685.c (I2C burst write to PWM chip) | |-- ps2.c
(SPI polling, analog-enable) | +-- comms.c (UART framing, CRC8 check) |-- control/ | |-- ik.c
(closed-form IK math) | |-- gait.c (tripod / ripple / wave) | +-- servo_mgr.c (angle limits,
channel map) +-- app/ |-- hexapod.c (state: IDLE / MANUAL / AUTO) +-- safety.c (watchdog, low-
battery)
```

Raspberry Pi / ROS2 Code

```
hexapod-rpi/ |-- ros2_ws/src/ | |-- hexapod_bridge/ (UART <-> ROS2 message bridge) | |--
vision_node/ (camera + obstacle detection) | +-- nav_node/ (path plan, sends commands to STM32)
|-- scripts/ | |-- calibrate.py (per-servo trim offset tool) | +-- tune_gait.py (live gait
parameter tuning) +-- config/ +-- robot.yaml (leg lengths, servo limits)
```

9 Testing & Debugging

Follow this 7-step sequence in order. Each step must pass before moving to the next. A problem caught at Step 2 takes minutes to fix. The same problem found at Step 7 takes days.

Step 1	Bench power only (no battery) PowerSTM32frombenchsupplyat 5V, 300mA. Flash blink-LED firmware. Confirm programming works.
Step 2	Single servo test Connectoneservoto PCA9685 CH0. Sweep 0-180 degrees. Oscilloscope: 50Hz, 1.0-2.0ms pulse.
Step 3	All 18 channels at neutral Setallchannelsto90deg(neutral). Each servo should hold position quietly. No twitching.
Step 4	PS2 controller test PrintrawSPIbytestoUSB. Press all buttons. Enable analog mode. Confirm joystick gives 0-255.
Step 5	IK math check RunIKwithknown inputs. Foot at home position should give approx 90, 90, 90 degrees.
Step 6	Single leg gait Onelegswingsthrough a complete cycle. Foot lifts cleanly, moves forward, lands gently.
Step 7	Full tripod gait All6legs.Startat smallest step size. Confirm stable standing. Increase step gradually.

Common Problems and Fixes

Symptom	Cause	Fix
Servos twitch or jitter	I2C noise / long cables	Shorten cables; add 100nF cap at PCA9685 VCC; use 4.2k pull-ups
STM32 resets while walking	6V rail droops from inrush	Add 4 x 1000uF caps on 6V rail; check star-point ground
Robot walks in circles	Servo trim not calibrated	Run <code>calibrate.py</code> ; measure physical leg lengths
UART errors (RPI to STM32)	Ground offset between boards	Check star-point ground; reduce baud rate first
IK produces wrong angles or NFAoNot outside reachable range		Add boundary check before IK; if D > femur+tibia, reject target
Servos do not move at all	PCA9685 OE pin not pulled LOW	Wire OE to GND, or pull LOW in firmware init
PS2 joystick stuck at centre	Analog mode not enabled	Send analog-enable command (0x44) on startup

10 Team Structure & Assignments

The team has 6 members. The Team Lead owns the STM32 + Zephyr firmware. Each member owns a specific domain and is responsible for delivering working, tested output — not just writing code.

LEAD: STM32 + Zephyr Firmware [TEAM LEAD]

Software: 80%

Hardware: 20%

Controls entire robot system. Handles all real-time execution.

- Set up Zephyr RTOS project on STM32 Nucleo
- Write PCA9685 I2C driver — controls all 18 servos
- Build 5 RTOS threads with correct priorities
- Implement double-buffer for jitter-free servo updates
- Write UART communication protocol to Raspberry Pi
- Handle emergency stop, watchdog, and low-battery shutdown

Deliverable: All 18 servos update at stable 50 Hz with no jitter

Member 1: Hardware & Power

Software: 20%

Hardware: 80%

Builds the robot. Responsible for reliable power to all parts.

- Assemble chassis and mount all 18 servo motors
- Build power board: LiPo + 5V buck + 6V buck
- Add 4 x 1000uF capacitor bank on 6V servo rail
- Wire all 18 servos to PCA9685 channels with labels
- Set up star-point common ground across all boards
- Measure and document each leg's physical dimensions (mm)

Deliverable: Robot powered on — both rails stable under full servo load

Member 2: IK & Gait Engine

Software: 90%

Hardware: 10%

Writes the movement math. Responsible for smooth, stable walking.

- Implement closed-form IK solver (atan2 + law of cosines)
- Write unit tests for IK: verify 10 known positions
- Implement tripod gait state machine
- Implement smooth foot arc trajectory (half-sine swing)
- Tune step height, step length, and body height
- Add ripple and wave gait with PS2 mode switching

Deliverable: Robot walks forward, backward, and turns under PS2 control

Member 3: Raspberry Pi & AI

Software: 90%

Hardware: 10%

Sets up RPi with camera, AI, and autonomous navigation.

- Install Ubuntu + ROS2 Humble on Raspberry Pi 4
- Write UART bridge: STM32 commands <-> ROS2 topics
- Set up Pi Camera v2 and OpenCV video pipeline
- Deploy YOLOv5-nano model for obstacle detection
- Write navigation node: sends avoid-commands to STM32
- Set up SSH access and live gait tuning via terminal

Deliverable: Camera detects obstacle; RPi sends stop command to STM32

Member 4: Communication & Integration

Software: 70%

Hardware: 30%

Owens the UART link. Makes STM32 and RPi work together.

- Define UART frame format: structure and CRC error check
- Test UART: confirm commands are received and executed
- Write calibrate.py: per-servo trim offset tool
- Build end-to-end integration test: PS2 -> STM32 -> servo
- Handle edge cases: RPi disconnect, bad packets, timeout
- Document all message types and byte format

Deliverable: End-to-end test: RPi command -> correct servo movement

Member 5: Testing & Debugging

Software: 60%

Hardware: 40%

Runs all tests, documents every issue, prepares the final demo.

- Execute 7-step bring-up checklist; sign off each step
- Write automated IK validation test with pass/fail output
- Test gait on 3 surfaces: smooth floor, tile, carpet
- Maintain issue log: failure, root cause, and fix
- Test E-stop, autonomous mode, and all gait switches
- Prepare and run final demo: walk + obstacle + PS2 override

Deliverable: Complete working system demonstrated on demo day

Responsibility Matrix

R=doesthework A=ownstheoutcome C = consulted I = informed

Task	LEAD	M1	M2	M3	M4	M5
Power system assembly	I	R/A	I I	I I I	I I I	C C
PCA9685 I2C driver	R/A	C	C C	I I I	I I I	C C
RTOS thread setup	R/A	I	R/A	C	R/A	C C
PS2 SPI driver	R/A	I	R/A	R/A	C	C C
IK solver	C	I	I I	C C	R/A	C
Gait state machine	C	I	C C	C	C C	R/A
UART protocol	R	I	C			R/A
ROS2 vision node	I	I				
Calibration scripts	C	C				
Integration testing	C	C				
Final demo	C	C				

11 Execution Plan

Five phases over 9 weeks. Each phase ends with a gate review — a live physical test that the work actually functions. Do not start the next phase until the current gate passes.

Phase 1: Hardware Setup	Weeks 1 - 2
● LEAD: Flash STM32, verify Zephyr builds, single servo sweep ● M1: Assemble chassis, mount all 18 servos, label wiring ● M1: Build power board (LiPo + bucks + cap bank + star ground) ● M5: Probe all 18 PCA9685 channels with oscilloscope	
Deliverable: All 18 servos hold neutral position — no voltage sag under load	
Phase 2: Servo Control	Weeks 2 - 3
● LEAD: Complete PCA9685 burst-write driver, 5-thread RTOS skeleton ● LEAD: PS2 SPI driver with analog-enable sequence ● M2: servo_mgr module — per-servo limits, direction inversion ● M4: UART frame format defined and tested end-to-end	
Deliverable: PS2 joystick drives any single joint angle in real time	
Phase 3: IK Implementation	Weeks 3 - 4
● M2: IK solver implemented and unit tested (10 known positions) ● LEAD: IK thread integrated with double-buffer angle updates ● M4: Serial command drives foot to 3D target position ● M5: Verify IK output physically matches commanded position	
Deliverable: Move any foot to a commanded XYZ position via serial terminal	
Phase 4: Walking	Weeks 4 - 6
● M2: Tripod gait with PS2 velocity input ● M2: Smoothing filter, tuned step height and step length ● M2: Ripple and wave gaits, PS2 mode switching ● M5: Gait tests on flat floor, tile, and carpet — log all issues	
Deliverable: Robot walks forward, backward, and turns under PS2 control	
Phase 5: AI Integration	Weeks 6 - 9
● M3: RPi + ROS2 setup, UART bridge to STM32 confirmed working ● M3: Pi Camera + YOLOv5-nano obstacle detection running ● M3: Navigation node sends avoid-commands to STM32 ● M4: Full integration test: RPi command -> robot movement ● M5: Final demo — autonomous walk + obstacle avoidance + PS2 override	
Deliverable: Robot walks autonomously, detects obstacles, avoids them	

9-Week Schedule

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9
Chassis + power (M1)	X	X							
STM32 + PCA9685 (LEAD)	X	X	X						
PS2 driver IK (LEAD)		X	X						
solver Gait (M2)			X	X					
engine Gait (M2)				X	X				
tuning RPi + (M2+M5)					X	X			
ROS2 Vision (M3)						X	X		
+ AI (M3)							X	X	
Integration (M4+M5)								X	X

Final Rule: Gate reviews are live physical demonstrations only — code on a screen does not count. If the hardware does not work correctly, the phase is not complete. Build strictly in order.